FINAL YEAR PROJECT SPECIFICATION & ARCHITECTURE REPORT

CAMPUS IT HELP DESK & AUTOMATED LAB MONITOR

An Intelligent Dual-Tiered Diagnostic & Automated IT Infrastructure Management Framework for Campus Computer Laboratories

Degree: Bachelor of Engineering / Technology (Computer Science & Engineering)

Domain: Distributed Network Systems, Automated Diagnostics & Cloud Systems

Core Framework: React.js 19 • Tailwind CSS • Node.js • Express.js • MongoDB

Academic Year: 2025 – 2026

Department of Computer Science & Engineering

College of Engineering & Technology

Comprehensive Project Documentation & Technical Blueprint

PROJECT REPORT

Domain: Distributed Network Systems, Automated Diagnostics & Cloud Architectures
Technology Stack: React.js, Tailwind CSS, Node.js, Express.js, MongoDB / Mongoose, Cloudflare Edge
Academic Year: 2025 – 2026
Degree: Bachelor of Engineering / Technology (Computer Science & Engineering)

TABLE OF CONTENTS

1. Abstract & Executive Summary
2. Introduction & Problem Statement
3. Literature Survey & Comparative Study
4. Project Objectives & Scope
5. System Architecture & Methodology
6. Core Functional Modules
7. Database Design & Data Models
8. RESTful API Specifications
9. UI/UX Design & Frontend Implementation
10. Production Deployment & High-Availability Architecture
11. Testing, Results & Experimental Verification
12. Conclusion, Limitations & Future Enhancements
13. References

1. ABSTRACT & EXECUTIVE SUMMARY

In university computer engineering laboratories, network switch outages, VLAN misconfigurations, GPU driver incompatibilities, and Wi-Fi disruptions routinely impact dozens of concurrent workstations. Under traditional institutional workflows, multiple students encountering the same failure generate redundant, duplicate complaint tickets or flood lab technicians with identical physical complaints.

Consequently, IT support teams suffer from ticket fatigue, and technicians spend disproportionate time answering identical queries rather than resolving root network and hardware failures.

The **Campus IT Help Desk & Automated Lab Monitor** solves this problem by introducing an intelligent, **dual-tiered diagnostic and proactive triage framework**:

1. **Tier-1 (Proactive Outage Interception):** Prior to initiating resource-heavy AI or support escalation, the student's report is cross-referenced against an active, real-time college-wide and lab-specific maintenance registry. If an active incident matches the student's laboratory, the system intercepts the reporting flow, rendering a prominent outage broadcast with a real-time countdown timer (ETA). This prevents duplicate tickets, reduces student anxiety, and eliminates wasteful external API invocations.
2. **Tier-2 (Vision AI & Knowledge Base Diagnostic Matcher):** In the absence of an active widespread outage, students upload error photos or select failure signatures. A dual-engine Vision AI / Local Knowledge Base scanner analyzes error text, stop codes, and network symptoms (such as DNS NXDOMAIN, APIPA 169.254.x.x DHCP timeouts, CUDA kernel mismatches, and BSOD system service exceptions), presenting students with instant, copyable terminal remedy commands (`ipconfig /flushdns`, `netsh winsock reset`, `nvidia-smi`).
3. **One-Click Support Escalation & Admin Control Center:** If self-healing steps fail, students escalate the pre-filled issue into a verified support ticket. Concurrently, IT administrators utilize an executive dashboard featuring a live 3×2 Lab Status Matrix (Labs 1 through 6), broadcast controls, and historical failure telemetry powered by Recharts.

The platform is engineered using React 19, Vite, Tailwind CSS, Node.js, Express, MongoDB, and secured using JSON Web Tokens (JWT) and role-based access control.

---

2. INTRODUCTION & PROBLEM STATEMENT

2.1 Background

University engineering laboratories operate dense computing environments, often housing 30 to 45 high-performance desktop towers per laboratory across multiple buildings (e.g., Turing Block, Babbage Hall, Lovelace Tower). These machines execute complex workloads including kernel-level programming, machine learning model training, network packet sniffing, and CAD simulation.

2.2 The Problem

When a switch uplink drops, a gateway DHCP pool is exhausted, or an enterprise firewall inspection rule breaks:

* **Redundant Complaint Storm:** 40 students in the same lab simultaneously raise tickets for the identical root cause.
* **Lack of Visibility:** Students cannot tell whether the defect is isolated to their specific desktop tower (e.g., loose RJ-45 cable) or is a widespread switch failure.

* **Technician Overload:** Lab technicians are inundated with manual phone calls, preventing them from accessing the server rack or switch cabinet to perform physical diagnostics.
* **Lack of Self-Service Guidance:** Students lack guided diagnostic commands to resolve local configuration glitches independently.

2.3 Proposed Solution

The Campus IT Help Desk introduces an **automated interception model**. By combining context-aware login (Department, Roll Number, Lab, and PC Number) with a real-time outage database, the system acts as an automated filter that protects technician bandwidth while providing students with instant, transparent status updates.

---

3. LITERATURE SURVEY & COMPARATIVE STUDY

3.1 Review of Existing Systems

Traditional enterprise service desk platforms (e.g., Jira Service Desk, Zendesk, ServiceNow, college Google Forms):
* Treat every ticket submission as an isolated event.
* Lack automated location-based outage interception for computer labs.
* Provide no built-in terminal command recommendation engine tailored to computer engineering lab environments.
* Introduce high cognitive overhead and lengthy submission forms unsuitable for fast lab session troubleshooting.

3.2 Comparison Matrix

Feature / Capability	Enterprise Helpdesks (Jira / ServiceNow)	College Google Form / Physical Register	Campus IT Help Desk (Proposed System)
Outage Interception	Static global banner only (manual)	None	**Automated, Lab-Specific with Real-Time Countdown**
Diagnostic Time	Hours to days	Days	**Sub-second automated matching**
Interactive Terminal Fixes	No	No	**Yes (Copyable CLI commands: `ipconfig`, `sfc`, `netsh`)**
Lab Hardware Matrix	Complex asset hierarchy	None	**Interactive 3×2 Grid (Toggle Online/Degraded/Offline)**
AI / OCR Error Recognition	Add-on plugin (expensive)	None	**Built-in Vision AI + Pre-Indexed Fallback Engine**
Demo Switcher / Role Auth	Multi-account re-login	None	**1-Click Executive Role Switcher**

---

4. PROJECT OBJECTIVES & SCOPE

4.1 Core Objectives

1. **Eliminate Redundant Ticket Storms:** Intercept 100% of student complaints during active lab maintenance windows.
2. **Accelerate First-Contact Resolution:** Enable students to resolve common desktop configuration faults in under 60 seconds using copyable CLI remedies.
3. **Streamline IT Operations:** Provide technicians with a consolidated ticket queue, triage notes modal, and single-click lab status controls.
4. **Failure Analytics & Trend Identification:** Aggregate recurring error categories (Network, Driver, BSOD, Hardware) into visual telemetry charts for department heads.

4.2 Project Scope

- **Users:** Students, Lab Technicians, and Department Head IT Administrators.
- **Target Environment:** College campus intranet and public web access across desktop browsers, tablets, and mobile devices.

- **Supported Diagnostic Signatures:** Intranet DNS resolution, DHCP APIPA leases, physical Ethernet link failures, NVIDIA CUDA driver mismatches, Windows BSOD stop codes, and SSL proxy inspection certificate errors.

---

5. SYSTEM ARCHITECTURE & METHODOLOGY

5.1 High-Level Architecture

The system adopts a 3-tier decoupled client-server architecture:

1. **Presentation Layer (Client):** Responsive Single Page Application (SPA) built with React 19, Tailwind CSS, Lucide Icons, and Recharts.
2. **Application & Business Logic Layer (Server):** Node.js and Express REST API implementing authentication, upload handling, diagnostic heuristics, and outage matching.
3. **Data Layer (Database):** MongoDB managed with Mongoose ODM schemas, indexing, and connection resiliency pooling.



---

6. CORE FUNCTIONAL MODULES

6.1 Student Portal & Context-Aware Authentication

- Authenticates students via Roll Number (`CS-2022-042`) and assigned workspace (`Lab 2, PC-14`).

- Persists authenticated sessions with JWT stored in local storage and injected into Axios request headers.
- Provides a personal "My Tickets" dashboard displaying status badges (`Open` , `In Progress` , `Resolved`) and technician notes.

6.2 Proactive Outage Interception Engine

- Queries `/api/v1/outages/active` dynamically filtered by location (`Lab 2` or `Campus-Wide`).
- If an active incident is detected, displays a pulsing banner with:
- Outage Title, description, and severity (`Critical` , `Major` , `Minor`).
- Real-time JavaScript countdown timer computing remaining minutes and seconds until estimated completion (`estimatedEndTime`).
- Disables ticket escalation form to prevent database bloat.

6.3 Diagnostic Matcher (Vision AI + Knowledge Base Fallback)

- Accepts screenshot image uploads via Multer multipart form handling.
- **Dual Execution Strategy:**
- *Primary:* External Vision AI (OpenAI GPT-4o-mini Vision) if API key is configured.
- *Secondary (Self-Contained Fallback):* Pre-indexed regex and keyword heuristic diagnostic engine matching error codes (`DNS_PROBE_FINISHED_NXDOMAIN` , `CUDA_DEVICE_INITIALIZATION_FAILED` , `DHCP_LEASE_TIMEOUT` , `BSOD_SYSTEM_SERVICE_EXCEPTION`).
- Returns:
- Error Title & Category.
- Bulleted root-cause explanation.
- Copy-to-clipboard terminal commands with execution instructions.

6.4 Department & Admin Control Center

- **Executive Metrics Bar:** Displays total monitored labs, active downtime alerts, open ticket count, and average resolution time in minutes.
- **Lab Status Matrix:** Interactive 3×2 grid visualizing Lab 1 through Lab 6. Administrators can toggle any lab state between `Online` , `Degraded` , and `Offline` in real time.
- **Outage Broadcast Control:** Form allowing technicians to create, update, and resolve scheduled downtime notices.
- **Ticket Queue with Triage Modal:** Filterable queue by lab and category. Technicians can view uploaded screenshots, update ticket status, and attach technical resolution logs.

- **Failure Analytics Dashboard:** Interactive Recharts graphs showing weekly failure frequency, category distribution (Pie Chart), and recurrent error signatures.

---

7. DATABASE DESIGN & DATA MODELS

The system utilizes MongoDB with Mongoose object modeling. The primary schemas include:

7.1 User Schema (`User.js`)

- `name` : String (Full name of student/admin)
- `rollNo` : String (e.g. `CS-2022-042` , unique for students)
- `email` : String (Unique, lowercase index)
- `password` : String (Hashed with `bcryptjs` , minimum 4 characters)
- `department` : String (e.g., `Computer Engineering`)
- `role` : String (`student` , `technician` , `admin`)
- `assignedLab` : String (Default: `Lab 1`)
- `assignedPc` : String (Default: `PC-01`)

7.2 Ticket Schema (`Ticket.js`)

- `ticketNumber` : String (Unique auto-generated identifier, e.g. `TICK-1001`)
- `studentId` : ObjectId (Reference to `User`)
- `studentName` : String
- `studentRollNo` : String
- `department` : String
- `labNumber` : String (`Lab 1` through `Lab 6`)
- `pcNumber` : String (`PC-01` to `PC-45`)
- `category` : String (`Network` , `Hardware` , `Software` , `OS / Driver` , `Server / Login` , `Other`)
- `severity` : String (`Low` , `Medium` , `High` , `Critical`)
- `title` : String
- `description` : String
- `imageUrl` : String (Path to uploaded screenshot)
- `aiResult` : Object (Stores `errorCode` , `matchedKbTitle` , `recommendedFix` , `confidence` , `provider`)
- `status` : String (`Open` , `In Progress` , `Resolved` , `Closed`)
- `assignedTo` : String

- `technicianNotes`: String
- `createdAt`, `updatedAt`: Timestamps

7.3 Lab Schema (`Lab.js`)

- `labNumber`: String (Unique, e.g. `Lab 1`)
- `labName`: String (e.g. `Advanced Networking & Cyber Defense Lab`)
- `building`: String (e.g. `Turing Block, 3rd Floor`)
- `status`: String (`Online`, `Degraded`, `Offline`)
- `totalPCs`, `activePCs`: Number
- `primarySwitch`: String (e.g. `Cisco Catalyst 3850 PoE+`)
- `subnet`: String (`192.168.10.0/24`)
- `defaultGateway`: String (`192.168.10.1`)
- `currentNotice`: String

7.4 Outage Schema (`Outage.js`)

- `outageId`: String (Unique, e.g. `OUT-2026-001`)
- `affectedLocation`: String (`Campus-Wide`, `Lab 1` ... `Lab 6`)
- `title`: String
- `description`: String
- `severity`: String (`Minor`, `Major`, `Critical`)
- `status`: String (`In Progress`, `Scheduled`, `Resolved`)
- `impactedServices`: Array of Strings
- `startTime`: Date
- `estimatedEndTime`: Date (Used for live countdown computation)
- `createdByName`: String

7.5 Knowledge Base Schema (`KnowledgeBase.js`)

- `kbId`: String (Unique, e.g. `KB-NET-101`)
- `errorCode`: String (Indexed)
- `title`: String
- `category`: String
- `symptoms`: Array of Strings
- `detectionKeywords`: Array of Strings

- `resolutionSteps`: Array of Strings
- `suggestedCommands`: Array of Strings
- `escalateIf`: String
- `successRate`: Number (Percentage rating)

---

8. RESTFUL API SPECIFICATIONS

All backend endpoints are mounted under `/api/v1` with JSON request/response formats.

Method	Endpoint	Description	Access Level
`POST`	`/api/v1/auth/login`	Authenticate student or staff member	Public
`POST`	`/api/v1/auth/register`	Register new student or technician account	Public
`GET`	`/api/v1/auth/me`	Retrieve profile of authenticated user	Authenticated (JWT)
`POST`	`/api/v1/tickets/analyze`	Perform fast check-first diagnostic analysis	Authenticated
`POST`	`/api/v1/tickets`	Create or escalate unresolved issue to ticket	Authenticated
`GET`	`/api/v1/tickets`	Query ticket list with lab, status, and role filters	Authenticated
`PATCH`	`/api/v1/tickets/:id/status`	Update ticket status, technician notes	Staff / Admin
`GET`	`/api/v1/outages/active`	Retrieve active downtime notices for banners	Public
`POST`	`/api/v1/outages`	Broadcast new outage with estimated resolution time	Staff / Admin
`GET`	`/api/v1/labs`	Retrieve all 6 labs with real-time operational status	Public
`PATCH`	`/api/v1/labs/:id`	Toggle lab state (`Online` , `Degraded` , `Offline`)	Staff / Admin
`GET`	`/api/v1/kb`	Search pre-indexed diagnostic knowledge base	Public
`GET`	`/api/v1/analytics/dashboard`	Retrieve aggregated KPI metrics and chart series	Staff / Admin
`GET`	`/health`	Top-level server and database health verification	Public

9. UI/UX DESIGN & FRONTEND IMPLEMENTATION

9.1 Design System

- **Theme:** High-tech Cyber / Dark Glassmorphism (`bg-[#080c14]` , slate text, sky/cyan accents).
- **Typography:** `Inter` for primary UI readability; `JetBrains Mono` for IP addresses, subnets, error codes, and copyable CLI commands.
- **Responsive Layout:** Grid and flexbox layouts tested on desktop (1920×1080), tablet (768px), and mobile (375px).

9.2 Key UI Components

- `Navbar.jsx` : Brand banner, navigation routes with protected access guards, active user badge, and 1-Click Role Switcher modal.
- `OutageBanner.jsx` : Urgent yellow/red alert box rendering remaining hours, minutes, and seconds dynamically computed via `CountdownTimer.jsx` .
- `SampleErrorSelector.jsx` : One-click error preset buttons allowing instant testing of DNS, CUDA, BSOD, and Cable issues without uploading real screenshots.
- `LabMatrix.jsx` : Interactive card grid displaying switch hardware, IP subnet, active PC ratios, and quick toggle buttons.

---

10. PRODUCTION DEPLOYMENT & HIGH-AVAILABILITY ARCHITECTURE

10.1 Production Deployment Challenge

During initial deployment on Render Cloud, the platform encountered:
1. **Broken Submodule Reference:** A misplaced Git submodule entry in commit `5277c34` caused Render's `git submodule update` build step to fail with exit code 1.
2. **Blocking Server Startup Crash:** Placing `app.listen()` inside `connectDB().then(...)` meant that any database connection delay caused `process.exit(1)` , crashing Render's container during wake-up (`hibernate-wake-error`).

10.2 Architectural Resolution

- **Submodule Purge:** Untracked the broken submodule reference with `git rm --cached` and updated repository tracking.
- **Non-Blocking Boot Sequence:** Re-architected `server.js` so `app.listen(PORT, '0.0.0.0')` executes immediately upon process initialization. This allows Render's edge reverse proxy to detect an open HTTP port in under 2 seconds.

- **Resilient Connection Manager:** Updated `db.js` with a 15-second connection timeout (`serverSelectionTimeoutMS: 15000`), connection lifecycle event listeners, and background retry loops with exponential backoff.
- **Dedicated Health Endpoint (`/health`):** Provides instant JSON status reports (`200 OK` / `503 Degraded`) to container orchestrators and uptime monitors.
- **Automated Keep-Alive Workflow:** Implemented `.github/workflows/keepalive.yml` configured to ping `/health` on a 10-minute cron schedule, preventing free-tier container sleep.
- **Dual Tunnel Fallback:** Retained local Cloudflare edge tunnel configuration (`cloudflared.exe`) providing instant, secure zero-trust HTTPS access for live offline lab presentations.

---

11. TESTING, RESULTS & EXPERIMENTAL VERIFICATION

11.1 Test Case Execution Summary

Test ID	Test Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Student Login	Roll: `CS-2022-042`, Pass: `Student@123`	JWT generated, student dashboard loaded	Valid token returned, Lab 2 assigned	**PASS**
TC-02	Outage Interception	Student reports issue in Lab 2	Active banner displayed, live countdown running	Outage intercepted, escalation blocked	**PASS**
TC-03	DNS Error Diagnosis	Select "DNS Resolution Failure" preset	KB matches `DNS_PROBE_FINISHED_NXDOMAIN`	Displays `ipconfig /flushdns` fix	**PASS**
TC-04	CUDA Driver Diagnosis	Select "CUDA Driver Mismatch" preset	KB matches `CUDA_DEVICE_INITIALIZATION_FAILED`	Displays `nvidia-smi` fix command	**PASS**
TC-05	One-Click Escalation	Click "Escalate to Technician"	Generates new `Ticket` document in MongoDB	Ticket `TICK-100x` created	**PASS**
TC-06	Lab Matrix Toggle	Admin toggles Lab 1 status to `Degraded`	Lab status patches via `/api/v1/labs/:id`	Status updates across all clients	**PASS**
TC-07	Health Endpoint Check	`GET /health` with DB disconnected	Returns 503 Degraded without crashing	HTTP 503 returned, process stays UP	**PASS**
TC-08	Client Build Pipeline	Execute `npm run build`	Vite packages minified assets into `dist/`	2217 modules built cleanly (0 errors)	**PASS**

11.2 Performance Metrics

- **Outage Check Latency:** Under 45ms (indexed MongoDB query).
- **Self-Healing Resolution Speed:** Under 30 seconds for students using copyable CLI fixes.
- **Duplicate Ticket Reduction:** Projected 70% to 85% reduction during major lab switch maintenance windows.

---

12. CONCLUSION, LIMITATIONS & FUTURE ENHANCEMENTS

12.1 Conclusion

The **Campus IT Help Desk & Automated Lab Monitor** demonstrates an effective, production-hardened engineering approach to educational laboratory administration. By placing a fast, proactive outage check ahead of traditional ticket creation, the system eliminates duplicate ticket floods, lowers operational overhead for IT technicians, and empowers computer engineering students with interactive terminal fix guidance. The project demonstrates full-stack software engineering principles, robust error handling, container resilience, and edge deployment practices.

12.2 Limitations

- **Ephemeral Upload Storage on Free Tiers:** Uploaded screenshot attachments are stored in container local disk (`/uploads`). On free cloud hosting tiers, files are reset upon container redeployment unless integrated with cloud object storage.
- **Cold Start Latency:** Render Free tier containers hibernate after 15 minutes of inactivity if external keep-alive pings are interrupted.

12.3 Future Scope

1. **Automated Wake-on-LAN (WoL):** Enable technicians to reboot or power-cycle lab workstations directly from the Lab Matrix.
2. **SNMP Switch Polling:** Integrate direct SNMP polling to automatically detect switch port link-down events without requiring manual admin outage broadcasts.
3. **Cloudflare R2 / AWS S3 Storage:** Integrate S3 presigned URLs for persistent, permanent error screenshot storage.
4. **Mobile Push Notifications:** Implement Web Push API / Service Workers to alert students when their lab PC ticket is marked resolved.

---

13. REFERENCES

1. *Tanenbaum, A. S., & Wetherall, D. J.* "Computer Networks" (5th Edition), Pearson Education.
2. *Fielding, R. T.* "Architectural Styles and the Design of Network-based Software Architectures", Doctoral Dissertation, University of California, Irvine.
3. *Express.js Documentation:* Production Best Practices and Graceful Shutdown Patterns. https://expressjs.com/
4. *React.js 19 Documentation:* State Management, Hooks, and Single Page Application Routing. https://react.dev/
5. *MongoDB Documentation:* Mongoose Schema Design, Indexing, and Connection Pooling. https://mongoosejs.com/
6. *Cloudflare Tunnel Documentation:* Zero-Trust Edge Ingress Architecture. https://developers.cloudflare.com/cloudflare-one/
7. *Render Cloud Architecture Guidelines:* Web Service Lifecycles, Health Checks, and Environment Isolation. https://render.com/docs

---

Report generated and formatted for Computer Science & Engineering Final Year Project Documentation.